Universidad Autónoma del Estado de Hidalgo

REPORTE DE PRÁCTICA NO. 4

ANÁLISIS LÉXICO DE PALABRAS

ALUMNO: Samuel Ortiz Salinas

Docente: Dr. Eduardo Cornejo-Velázquez

1. Identificación

- Nombre de la práctica: Análisis léxico de palabras
- No. de práctica: 4
- No. de sesiones: 2
- No. de integrantes máximo por equipo: 2

2. Introducción

El analizador léxico es la primera fase de un compilador. Su trabajo es leer los caracteres de entrada uno por uno y agruparlos en componentes léxicos (tokens), que son los que después usa el analizador sintáctico. A esta fase también se le llama "scanner". Mientras va leyendo, el analizador léxico también es el que detecta si aparece un carácter que no pertenece al alfabeto del lenguaje, o una cadena que no coincide con ningún patrón válido [1].
En la práctica 2 ya se trabajó reconociendo palabras de un lenguaje, pero ahí se hizo "a mano", definiendo el AFD completo con su tabla de transiciones. En esta práctica se usa una herramienta hecha justo para esto: **LEX** (en este caso su versión libre, **Flex**), que permite escribir las reglas léxicas directamente como expresiones regulares y genera automáticamente el código en C que las reconoce, sin tener que programar el AFD manualmente [2].

3. Marco teórico

Una **expresión regular** es una notación que describe un conjunto de cadenas de un alfabeto. Cada expresión regular tiene asociado un AFD que la reconoce, y eso es justo lo que hace LEX por debajo: toma las expresiones regulares que se le dan y construye el autómata que las reconoce, sin que el programador tenga que construirlo él mismo [1].
Un programa en LEX se divide en tres secciones separadas por `%%`:

1. **Definiciones**: aquí van los `#include` de C (entre `%{ %}`) y los nombres que se le dan a patrones que se van a reutilizar, por ejemplo `ENTERO [0-9]+`.
2. **Reglas**: cada línea es un patrón (una expresión regular, o un nombre definido entre llaves {...}) seguido de una acción en C entre llaves { }, que se ejecuta cuando ese patrón hace match. La variable `yytext` siempre tiene el texto que se reconoció.
3. **Procedimientos**: código C normal, como la función `main()` que llama a `yylex()` para iniciar el análisis, y `yywrap()` que indica qué hacer al llegar al final del archivo de entrada.

Cuando hay varias reglas que podrían reconocer el mismo texto, LEX sigue dos criterios para decidir cuál usar: primero gana la regla que reconozca la cadena **más larga** (maximal munch); y si dos reglas reconocen exactamente la misma longitud, gana la que esté escrita **primero** en el archivo `.l` [2].

4. Objetivo general

Construir programas en un lenguaje de programación de alto nivel que realicen el análisis de cadenas de símbolos que forman palabras, basado en la implementación de estructuras de datos y algoritmos de análisis, además del uso de expresiones regulares como segunda estrategia de implementación.

5. Objetivos específicos

- Identificar el proceso básico de análisis de cadenas de símbolos de un alfabeto.
- Implementar estructuras de datos y algoritmos de análisis de cadenas de símbolos en un lenguaje de programación de alto nivel.
- Implementar expresiones regulares para el análisis de cadenas de símbolos en un lenguaje de programación de alto nivel.

6. Equipo y software a utilizar

Equipo de cómputo con **Flex** (implementación libre de LEX) y **GCC** instalados. El programa se escribe en un solo archivo `.l`, se traduce a C con `flex` y se compila con `gcc`, todo desde la terminal.

7. Desarrollo

Procedimiento

Siguiendo el procedimiento de la práctica, se construyeron reconocedores para 6 tipos de cadenas:

1. Números enteros.
2. Números decimales.
3. Variables (identificadores): inician con una letra y después pueden tener letras, números y guiones bajos _.
4. RFC.
5. CURP.
6. Correo electrónico institucional.

En vez de hacer 6 archivos `.l` por separado, se juntaron los 6 patrones dentro de un solo archivo (`analizador.l`), cada uno como una definición nombrada en la sección de definiciones y su propia regla en la sección de reglas.

Listing 1: Definiciones de los 6 patrones (analizador.l).

```
RFC         [A-Z&Ñ]{4}[0-9]{2}(0[1-9]|1[0-2])(0[1-9]|[12][0-9]|3[01])[A-Z0-9]{3}
CURP        [A-Z]{4}[0-9]{2}(0[1-9]|1[0-2])(0[1-9]|[12][0-9]|3[01])[HM][A-Z]{5}[0-9A-Z][0-9]
EMAIL       [a-zA-Z0-9_.+-]+@([a-zA-Z0-9-]+\.)+edu\.mx
DECIMAL     [+-]?[0-9]+\.[0-9]+
ENTERO      [+-]?[0-9]+
VARIABLE    [a-zA-Z][a-zA-Z0-9_]*
```

El RFC se arma con 4 letras (iniciales del nombre y apellidos), 6 dígitos de la fecha de nacimiento (año, mes, día) y 3 caracteres de homoclave. La CURP es parecida pero con 18 caracteres: 4 letras, 6 dígitos de fecha, una letra de sexo (`H` o `M`), 5 letras más y un carácter y dígito finales. Para el correo institucional se pidió que el dominio termine en `.edu.mx`, permitiendo cualquier cantidad de subdominios antes (por ejemplo `alumnos.uabc.edu.mx`).

Listing 2: Reglas de reconocimiento, en orden (analizador.l).

```
{RFC}       { printf("RFC identificado: %s\n", yytext); }
{CURP}      { printf("CURP identificada: %s\n", yytext); }
{EMAIL}     { printf("Correo institucional identificado: %s\n", yytext); }
{DECIMAL}   { printf("Numero decimal encontrado: %s\n", yytext); }
{ENTERO}    { printf("Numero entero encontrado: %s\n", yytext); }
{VARIABLE}  { printf("Variable (identificador) encontrada: %s\n", yytext); }
[ \t\n]+    { /* ignorar espacios, tabulaciones y saltos de linea */ }
.           { printf("Caracter no reconocido: %s\n", yytext); }
```

El **orden de las reglas** se eligió a propósito. RFC y CURP se pusieron antes que VARIABLE porque una cadena como un RFC válido también cumple con el patrón de VARIABLE (empieza con letra y sigue con letras/números); como LEX reconoce primero la cadena más larga y, si hay empate de longitud, usa la regla que está más arriba, poner RFC y CURP antes garantiza que una cadena con ese formato se identifique como lo que realmente es y no como una variable cualquiera. Por la misma razón DECIMAL se puso antes que ENTERO, aunque aquí no hacía tanta falta porque un decimal siempre es más largo que el entero que hay al principio de él, y LEX de todas formas elige la coincidencia más larga.
Para los caracteres que no encajan en ningún patrón (como `@` suelto, puntuación, etc.) se dejó la regla `.` al final, que los reporta como "Carácter no reconocido" en vez de dejar que el programa truene.

Compilación

Compilación desde la terminal (sin interfaz gráfica):

```
flex analizador.l
gcc lex.yy.c
```

El comando `flex` traduce el archivo `.l` a un programa en C (`lex.yy.c`), y `gcc` lo compila al ejecutable por defecto `a.out`, que es el que se ejecuta con `./a.out`.

Ejecución y validación

El programa se ejecutó desde la terminal, dándole como entrada una línea de texto con ejemplos de los 6 tipos de cadena: una variable, un entero positivo y uno negativo, un decimal positivo y uno negativo, una CURP, un RFC y un correo institucional.

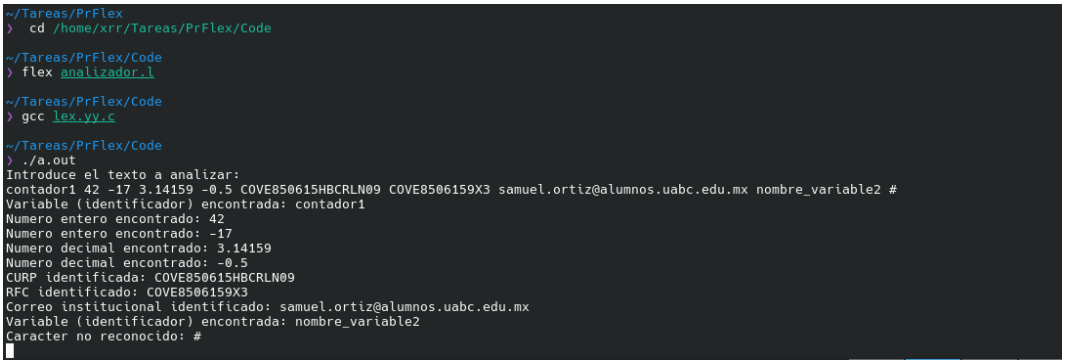


Figure 1: Terminal ejecutando `./a.out` con la entrada de prueba.

La salida real obtenida fue la siguiente:

Listing 3: Salida real del analizador con la entrada de prueba.

```
$ printf 'contador1 42 -17 3.14159 -0.5 COVE850615HBCRLN09 COVE8506159X3 samuel.
    ortiz@alumnos.uabc.edu.mx nombre_variable2 #\n' | ./a.out
Introduce el texto a analizar:
Variable (identificador) encontrada: contador1
Numero entero encontrado: 42
Numero entero encontrado: -17
Numero decimal encontrado: 3.14159
Numero decimal encontrado: -0.5
CURP identificada: COVE850615HBCRLN09
RFC identificado: COVE8506159X3
Correo institucional identificado: samuel.ortiz@alumnos.uabc.edu.mx
Variable (identificador) encontrada: nombre_variable2
Caracter no reconocido: #
```

Los 6 tipos de cadena se reconocieron correctamente, incluyendo el correo con varios subdominios antes de `.edu.mx`, y el carácter `#` que no pertenece a ningún patrón se reportó como no reconocido en vez de tronar el programa.
También se probó con una entrada de texto normal, para ver qué pasa con palabras comunes y con un correo que **no** es institucional:

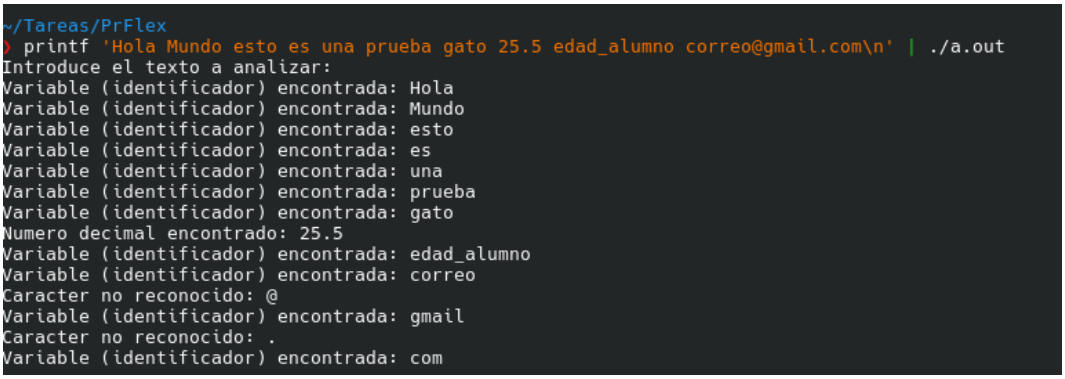


Figure 2: Terminal con texto libre y un correo no institucional.

Listing 4: Salida real del analizador con texto libre.

```
$ printf 'Hola Mundo esto es una prueba gato 25.5 edad_alumno correo@gmail.com\n' | ./a.out
Introduce el texto a analizar:
Variable (identificador) encontrada: Hola
Variable (identificador) encontrada: Mundo
Variable (identificador) encontrada: esto
Variable (identificador) encontrada: es
Variable (identificador) encontrada: una
Variable (identificador) encontrada: prueba
Variable (identificador) encontrada: gato
Numero decimal encontrado: 25.5
Variable (identificador) encontrada: edad_alumno
Variable (identificador) encontrada: correo
Caracter no reconocido: @
Variable (identificador) encontrada: gmail
Caracter no reconocido: .
Variable (identificador) encontrada: com
```

Aquí se ve que `correo@gmail.com` no se reconoce como correo institucional (porque no termina en `.edu.mx`), así que el analizador lo va partiendo en pedazos: reconoce `correo` como variable, el `@` y los puntos como caracteres no reconocidos, y `gmail` y `com` otra vez como variables. Esto confirma que el patrón de correo institucional sí está restringido al dominio que se pidió y no cualquier correo.

8. Resultados

Se Hizo en un solo archivo `.l` el léxico capaz de reconocer los 6 tipos de cadena pedidos (enteros, decimales, variables, RFC, CURP y correo institucional). En las pruebas realizadas, los 6 patrones identificaron correctamente las cadenas de ejemplo, y las cadenas con caracteres fuera de cualquier patrón (como `@` o `.` sueltos, o un correo no institucional) se reportaron como "Carácter no reconocido" sin detener la ejecución del programa completo.

9. Conclusiones

Esta práctica ayudó a ver la diferencia entre programar un analizador léxico "a mano" (como en la práctica 2, con la tabla de transiciones) y usar una herramienta como FLEX, que genera el reconocedor directamente a partir de las expresiones regulares. También aydo reconocer que el orden de las reglas si importa cuando dos patrones pueden reconocer la misma cadena (como RFC/CURP contra VARIABLE), porque FLEX usa primero la coincidencia más larga y, si hay empate, la regla que está más arriba en el archivo.

10. Cuestionario

1. **¿Cuál es la utilidad de las expresiones regulares en la identificación de palabras de un lenguaje?**

   Pues que con una expresión regular ya no hay que armar el AFD a mano para cada patrón, solo se describe cómo se ve la cadena y FLEX arma el reconocedor solo. Sirve para describir fácil cosas como un RFC o un correo sin tener que describir los estados enteros.

2. **¿Qué elementos del lenguaje de programación utilizaste para la implementación de las estructuras de datos?, ¿Cómo las utilizaste?**

   La verdad no usé estructuras de datos complicadas, porque LEX ya hace casi todo. Lo único que usé fue la variable `yytext` que da el texto que se reconoció y la función `printf` de C para mostrar qué tipo de cadena era cada cosa.

3. **¿Cuál es el proceso que seguiste para analizar las cadenas de símbolos?**

   Primero definí cada patrón como expresión regular (RFC, CURP, correo, decimal, entero, variable). Luego los puse en orden en las reglas, dejando RFC y CURP antes de variable para que no se confundieran. Después agregué una regla para ignorar espacios y otra al final con un punto para atrapar cualquier carácter raro. Ya con eso compilé con `flex` y `gcc` y fui probando con distintas cadenas de prueba.

4. **¿Qué elementos del lenguaje de programación te permitieron implementar expresiones regulares?, ¿qué condiciones o recomendaciones se deben seguir al escribir las expresiones regulares?**

   FLEX deja escribir las expresiones regulares directo en la sección de reglas, con cosas como `[A-Z]`, `{4}` para repetir, y `|` para varias opciones. Como recomendación, aprendí que hay que poner primero los patrones más específicos (como RFC o CURP) antes que los genéricos (como variable), si no el patrón genérico se los gana y nunca se reconocen bien.

Referencias Bibliográficas

References

[1] Aho, A. V., Lam, M. S., Sethi, R., Ullman, J. D. (**2007**). *Compilers: Principles, Techniques, and Tools*. Segunda edición. Editorial Pearson Addison-Wesley. U.S.A.

[2] Lesk, M. E., Schmidt, E. (**1975**). *Lex – A Lexical Analyzer Generator*. Bell Laboratories, Computing Science Technical Report No. 39. Murray Hill, New Jersey, U.S.A.